

BIND: Enabling Continuous Transaction Processing During Account Migration in Sharded Blockchains

Jiahao Qi
Shanghai Jiao Tong University
Shanghai, China
qjh2023@sjtu.edu.cn

Dian Ding*
Shanghai Jiao Tong University
Shanghai, China
dingdian94@sjtu.edu.cn

Jie Li
Shanghai Jiao Tong University
Shanghai, China
lijiecs@sjtu.edu.cn

Jiannong Cao
Hong Kong Polytechnic University
Hong Kong, China
csjcao@comp.polyu.edu.hk

Yi-Chao Chen
Shanghai Jiao Tong University
Shanghai, China
yichao@sjtu.edu.cn

Guangtao Xue
Shanghai Jiao Tong University
Shanghai, China
gt_xue@sjtu.edu.cn

Shengyun Liu
Shanghai Jiao Tong University
Shanghai, China
shengyun.liu@sjtu.edu.cn

Abstract

Account migration in sharded blockchains presents a critical trade-off between optimization effectiveness and system availability. While dynamically reallocating accounts across shards can significantly reduce cross-shard transaction overhead, existing migration mechanisms cause service disruptions that intensify as state data volumes grow. To address this challenge, we propose BIND, a batch-wise account migration protocol that eliminates service interruptions by enabling continuous transaction processing throughout migration. BIND introduces a dual transaction pool architecture that isolates transactions involving migrating accounts while allowing non-migrating accounts to operate uninterrupted. To optimize migration efficiency, we design a reverse greedy heuristic algorithm that partitions accounts into batches based on community cohesion, maximizing intra-batch connectivity to front-load cross-shard communication reduction. We evaluate BIND using real Ethereum transactions, demonstrating superior performance over existing mechanisms. BIND achieves 12% higher overall throughput, reduces migration time to 23.6%-39.3% of the one-shot baseline (across 1-10Gbps bandwidth), and lowers cross-shard transaction rates by 24.1% compared to random batching. These results confirm BIND as a practical solution for large-scale, non-disruptive account migration in production sharded blockchains.

CCS Concepts

• **Theory of computation** → Design and analysis of algorithms; • **Computer systems organization** → Dependable and fault-tolerant systems and networks; • **Networks** → Network architectures.

*Dian Ding is the corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792530>

Keywords

Blockchain, Sharding, Account Migration, Cross-Shard Transactions

ACM Reference Format:

Jiahao Qi, Dian Ding, Jie Li, Jiannong Cao, Yi-Chao Chen, Guangtao Xue, and Shengyun Liu. 2026. BIND: Enabling Continuous Transaction Processing During Account Migration in Sharded Blockchains. In *Proceedings of the ACM Web Conference 2026 (WWW '26), April 13–17, 2026, Dubai, United Arab Emirates*. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3774904.3792530>

1 Introduction

With the rapid advancement of blockchain technology, its widespread adoption in domains such as the Internet of Things (IoT) [22, 23, 27], decentralized finance (DeFi) [2, 25], mobile Internet [9, 17] and even in the AI field [8, 19–21] has demonstrated immense potential for revolutionizing digital transactions and distributed applications. However, as user bases expand exponentially, foundational single-chain systems such as Bitcoin and Ethereum increasingly encounter significant scalability bottlenecks. Sharding has emerged as one of the most promising solutions to this challenge, enhancing overall transaction processing capacity by partitioning the network into multiple independent sub-chains (i.e., shards), where each shard processes a distinct subset of transactions in parallel.

While sharding enables substantial performance gains through parallelization, it introduces a fundamental challenge: **cross-shard transactions**. These are transactions that involve accounts residing on different shards, requiring inter-shard communication and coordination to ensure atomicity and consistency. To mitigate the communication overhead and latency associated with cross-shard transactions, numerous studies have proposed account reallocation mechanisms that optimize account-to-shard mappings by co-locating frequently interacting accounts. However, existing research has largely focused on *generating* optimized allocation

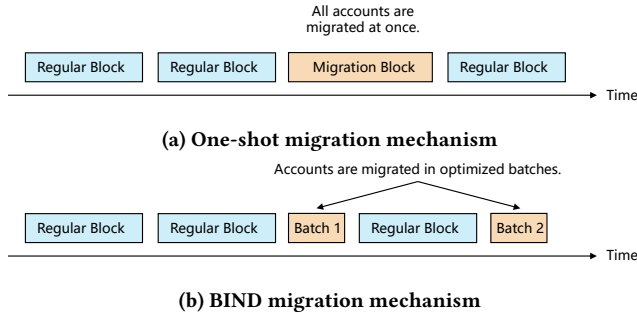


Figure 1: Comparison of Account Migration Mechanisms.

schemes, while a critical challenge remains largely overlooked: **ensuring the continuous availability and uninterrupted transaction processing of shards during the execution of the account migration process itself.**

Existing works, such as BrokerChain [6] and LB-Chain [15], typically adopt a straightforward one-shot migration approach, as illustrated in Figure 1a. In this model, all accounts designated for migration are transferred within a single, monolithic migration window. To prevent state inconsistencies and double-spending attacks during this critical period, the relevant accounts and transactions must be locked, which *severely interferes with normal transaction processing* until the migration completes. While this approach ensures atomicity, it results in significant service disruptions. The problem is further exacerbated by the increasing volume of account state data in real-world blockchain systems. For instance, as of September 2025, Ethereum hosts over 80 million smart contract accounts [3], many with substantial state data. Migrating such large-scale state in a single batch is not only impractical but would cause prolonged service outages that are unacceptable for latency-sensitive and mission-critical applications, such as financial services and supply chain management.

To overcome these fundamental limitations, we propose **BIND**, a novel account migration protocol designed to maintain high system availability and transaction throughput throughout the migration process. BIND's core innovation lies in its ability to **allow shards to process regular transactions concurrently with the migration of account state**, thereby eliminating the service disruptions inherent in traditional approaches. This is achieved through two synergistic mechanisms:

- **Batch-wise Migration Mechanism:** BIND divides the set of accounts designated for migration into multiple smaller, manageable batches. Instead of migrating all accounts in a single step, the protocol executes migration incrementally over a series of consecutive migration windows, with each window handling one batch. By reducing the scope and state volume of each migration step, BIND ensures that the consensus time for each migration block remains brief, comparable to that of a regular block, thereby preserving normal block generation cycles and system responsiveness.
- **Dual Transaction Pool Architecture:** To enable parallel transaction processing during migration, BIND introduces a novel dual transaction pool design. Transactions involving accounts in the current migration batch are isolated into

a separate, temporarily paused *migration transaction pool*, while all other transactions continue to be processed from the active *normal transaction pool*. This architectural innovation ensures that non-migrating accounts, which constitute the majority of accounts in the system, experience zero disruption, maintaining 100% transaction processing capability throughout the migration phase.

However, the batch-wise approach introduces a new challenge: **the optimization benefits of the new allocation scheme are realized incrementally rather than instantaneously.** A naive or random batching strategy could inadvertently separate highly interactive accounts into different batches, temporarily increasing cross-shard communication overhead during intermediate migration windows. To address this issue, we introduce a **reverse greedy heuristic algorithm** that intelligently partitions accounts based on *community cohesion*. The algorithm operates in reverse order by iteratively identifying and removing the least cohesive accounts and assigning them to later batches. This ensures that tightly-knit account communities, which generate the highest intra-community transaction volumes, remain intact and are scheduled for *earlier* migration. By front-loading the migration of high-cohesion communities, BIND maximizes the reduction of cross-shard communication costs from the very beginning of the migration process, effectively mitigating the delayed optimization effect inherent in phased migration.

To validate the effectiveness and practicality of BIND, we implemented **BINDChain** integrating all core components of the BIND protocol, including the dual transaction pool architecture, and the reverse greedy batching algorithm. We conducted extensive experiments using real historical transactions from the Ethereum mainnet, representing diverse transaction patterns and account interaction dynamics.

The main contributions of this paper are as follows:

- We propose **BIND**, a non-disruptive account migration protocol that employs a batch-wise mechanism and dual transaction pool architecture to maintain continuous transaction processing during migration. A reverse greedy algorithm optimizes batch composition to front-load cross-shard communication reduction.
- We implement **BINDChain**, a production-ready prototype integrating the BIND protocol, and provide a comprehensive description of its architecture and workflow, demonstrating practical feasibility.
- We conduct extensive experiments on real Ethereum data, showing BIND achieves 12% higher throughput, reduces migration time to 23.6%-39.3% of baseline, and lowers cross-shard transaction rates by 24.1% compared to random batching.

2 Related Work

2.1 Blockchain Sharding and Cross-Shard Transactions

Sharding has emerged as a fundamental scalability solution for blockchain systems, enabling parallel transaction processing by partitioning the network into multiple independent subsets (i.e.,

shards). Elastico [18] pioneered blockchain sharding by randomly assigning nodes to committees, achieving the first breakthrough in parallel transaction throughput. Building upon this, OmniLedger [11] integrated the Practical Byzantine Fault Tolerance (PBFT) consensus mechanism [1] and designed the Atomix protocol based on two-phase commit (2PC) to ensure atomicity in cross-shard transactions. RapidChain [28] optimized cross-shard communication protocols, while Co-Chain [14] proposed the Consensus on Consensus mechanism, improving concurrency performance of small-scale shards.

Despite these advances, cross-shard transactions—interactions involving accounts distributed across different shards—remain a critical performance bottleneck. To address this, X-Shard [26] adopted an optimistic concurrency model for parallel processing, while Monoxide [24] introduced a relay mechanism for efficient state synchronization. Additional studies, including Meepo [29], Pyramid [4], Sharon [10], and SPRING [16], have explored the balance between transaction parallelism and consistency. While these works improve cross-shard transaction processing efficiency, they do not fundamentally address the root cause: suboptimal account-to-shard mappings.

2.2 Account Migration and Reallocation

Account migration mechanisms focus on proactively reducing cross-shard transaction frequency by dynamically reallocating accounts across shards based on their interaction patterns. Early research primarily concentrated on generating optimized migration plans. Shard Scheduler [12] analyzed historical transaction data to identify high-frequency interacting account pairs and co-located them within the same shard to reduce cross-shard traffic. Li et al. [13] proposed the CLPA community detection algorithm, which strikes a balance between the dual objectives of minimizing cross-shard transactions and maintaining balanced shard loads. BrokerChain [6] leveraged a global account transaction graph to dynamically adjust account distributions, adapting to evolving transaction patterns in real-time.

However, existing research has largely overlooked the critical challenge of executing the migration process itself without disrupting system availability. BrokerChain, LB-Chain [15] and Fine-Tuned Lock [5] implemented a straightforward one-shot approach, where all accounts designated for migration are transferred within a single migration window. This requires inevitably halting all blockchain services during the migration period. Although the Fine-tuned Lock protocol enables the processing of partial transactions during the account migration phase, it requires special auxiliary transactions to complete the final synchronization of account states upon the conclusion of migration and cannot truly eliminate service interruptions.

3 BINDChain Design

To facilitate a clear description of the BIND protocol, we first introduce **BINDChain**, the prototype system we developed to implement and evaluate our design. This section details the system model, architecture, and operational workflow of BINDChain.

3.1 System Model

The BIND protocol is designed to be compatible with any existing blockchain sharding system, especially those that use account reallocation mechanisms to reduce cross-shard communication costs. To meet the complex requirements of smart contracts, we adopt an account-based sharding model. We assume that the blockchain network consists of multiple shards, each maintained and operated by a group of nodes. Each account is mapped to a specific shard, and all transactions related to that account are processed within that shard. For simplicity, we assume that each shard has the same number of nodes and that each shard uses the Practical Byzantine Fault Tolerance (PBFT) consensus algorithm. This consensus algorithm allows up to one-third of the nodes in each shard network to be malicious. We assume that the nodes in the shard network are honest but curious, meaning they follow protocol rules but may attempt to gain additional information by analyzing transaction data. We assume that nodes in the shard network can communicate through secure channels to prevent man-in-the-middle attacks and data tampering. We also assume that nodes in the blockchain network have sufficient computing power and storage capacity to process transactions and maintain the blockchain ledger.

3.2 Architecture of BINDChain

BINDChain adopts an architecture with one Beacon Shard and multiple transaction shards, which is compatible with most existing sharding systems. Each shard uses the PBFT protocol for consensus. The details of these shards are described in the following sections as shown in Fig.2.

3.2.1 Beacon Shard. The Beacon Shard serves as the central coordinator of the blockchain network, managing critical metadata such as shard statuses, account-to-shard mappings, and the orchestration of account migration. It periodically aggregates account activity and transaction statistics from all transaction shards to generate updated account mapping relationships, referred to as new allocation schemes. Based on these schemes, the Beacon Shard formulates batched account migration plans, which are then disseminated to the transaction shards. This ensures that the migration process is executed seamlessly, maintaining normal transaction processing and avoiding service disruptions.

3.2.2 Transaction Shards. Transaction shards handle user transaction requests and maintain the state of accounts assigned to them. To fully exploit the benefits of sharding, the network is divided into multiple transaction shards, each responsible for processing a distinct subset of transactions. Each transaction shard employs the PBFT protocol to achieve secure and reliable consensus on transaction processing. Periodically, transaction shards report account activity and transaction statistics to the Beacon Shard, which uses this data to generate updated account mappings and batched migration plans. Upon receiving these migration plans, transaction shards execute account migration during designated migration windows, ensuring seamless integration of the new allocation scheme.

3.3 Workflow of BINDChain

Before describing the workflow, we clarify key terminology: An *epoch* is a predefined time interval at the end of which the Beacon

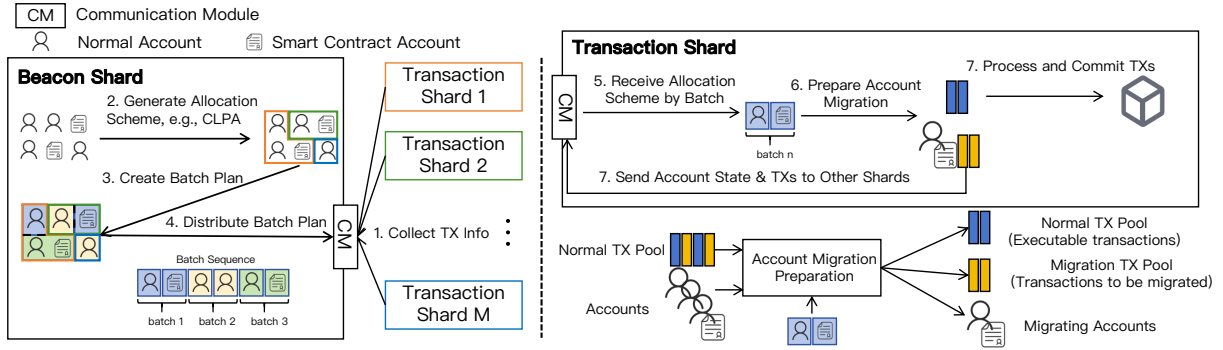


Figure 2: Overview of BINDChain.

Shard initiates reallocation. A *migration window* is a discrete period during which one batch of accounts is migrated via a special *migration block* that records state transitions and is finalized through PBFT consensus. A *regular block* refers to a block generated by packaging normal transactions processed during the non-migration phase.

Existing sharding systems concentrated account migration operations within a single migration window, leading to prolonged suspension of transaction processing and significant service disruptions. To overcome this limitation, BINDChain introduces a batched account migration mechanism that enables shards to continue processing transactions during migration windows, thereby ensuring high system availability.

The workflow of BINDChain is divided into three key stages: **allocation scheme generation, batched account migration plan generation, and account migration**. Unlike traditional approaches, the migration window in BINDChain is divided into multiple periods, with each batch of accounts corresponding to a distinct migration window. Notably, the allocation scheme and batched migration plan are generated only once, prior to the start of the first migration window. Each subsequent migration window focuses exclusively on migrating the accounts in the designated batch.

3.3.1 Allocation Scheme Generation. After each epoch, the Beacon Shard aggregates account activity and transaction statistics from all transaction shards to generate updated account-to-shard mappings, referred to as new allocation schemes. These schemes are designed to minimize cross-shard communication costs, thereby enhancing the overall transaction processing efficiency of the blockchain network. BINDChain is compatible with any existing account reallocation algorithm; for clarity, we adopt the CLPA algorithm in our implementation. CLPA is a community detection-based approach that effectively identifies interaction patterns among accounts, assigning frequently interacting accounts to the same shard. This strategy significantly reduces the number of cross-shard transactions, optimizing network performance.

3.3.2 Batched Account Migration Plan Generation. After generating the new allocation scheme, the Beacon Shard formulates a batched account migration plan based on the updated scheme. This process is a critical component of the BIND protocol. Specifically, the Beacon Shard partitions the accounts designated for migration

into multiple batches and selects a subset of accounts for each batch. This batched approach effectively reduces cross-shard communication costs during migration while allowing transactions unrelated to the migrating accounts to be processed without interruption. The details of the batched migration plan generation are provided in Section 4.

Once the batched migration plan is generated, the Beacon Shard distributes it, along with the complete allocation scheme, to the consensus nodes within the shard. Using the PBFT protocol, consensus is reached to ensure the validity and consistency of the migration plan. After achieving consensus, the migration plan and allocation scheme are disseminated to all transaction shards, initiating the account migration process.

3.3.3 Account Migration Stage. After receiving the new allocation scheme and batched account migration plan, transaction shards initiate the corresponding number of account migration windows based on the batch information and prepare for account migration, mainly dividing transactions into regular transactions and migrating transactions in accordance with the plan of the current batch. During each migration window, transaction shards migrate the accounts in the current batch and their transactions from the source shard to the target shard as specified in the batched migration plan. Specifically, transaction shards record the list of accounts to be migrated in the current batch along with their latest state information in migration blocks, which are finalized through the PBFT consensus protocol. Once consensus is reached, these migration blocks are appended to the shard's blockchain ledger, and the corresponding state information is updated in the world state maintained by the shard.

In parallel with the migration process, transaction shards continue processing transactions for non-migrating accounts. This is enabled by the dual transaction pool architecture introduced in the BIND protocol. While the migration transaction pool temporarily locks transactions involving migrating accounts, the normal transaction pool remains active, allowing shards to seamlessly process transactions unrelated to the migrating accounts without interruption.

4 BIND Protocol

In this section, we provide a comprehensive overview of the design of the BIND protocol. The BIND protocol is built around two

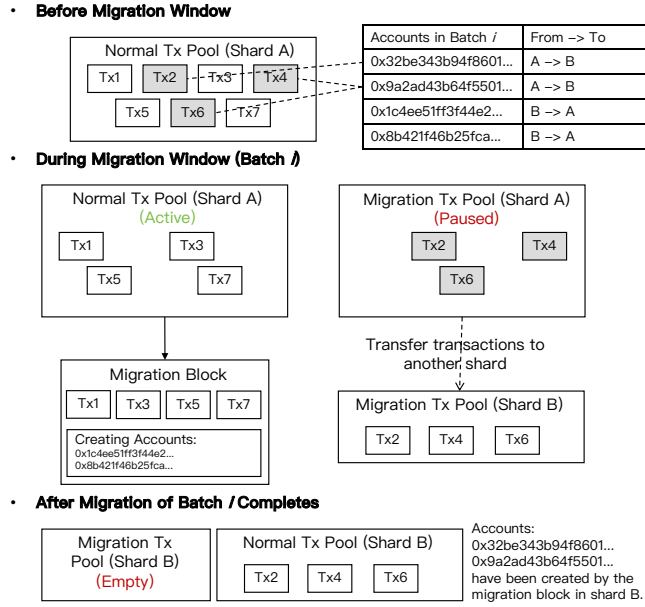


Figure 3: Illustration of dual transaction pool architecture during account migration between Shard A and Shard B.

core components: **transaction execution during the migration phase** and **batch-wise account migration scheme generation**.

4.1 Transaction Execution During the Migration Phase

The BIND protocol enables shards to continue processing transactions concurrently with account migration, thereby maintaining high system availability.

4.1.1 The Challenge: Transaction Pool Locking. Under normal circumstances, each node maintains a transaction pool to store pending transactions that have not yet been packaged into blocks. During an account migration window, the system must lock the portion of the transaction pool involved in migration. However, a blunt approach of locking the entire transaction pool would result in severe service disruption across the entire system, thus necessitating a specialized design for the transaction pool.

4.1.2 The Solution: Dual Transaction Pool Architecture. To address this challenge, the BIND protocol introduces a **dual transaction pool architecture**, as illustrated in Figure 3. This architecture consists of two independent transaction pools:

- **Normal Transaction Pool:** Stores transactions that do not involve any accounts designated for migration in the current batch. Transactions in this pool continue to be processed and packaged into regular blocks without interruption.
- **Migrated Transaction Pool:** Stores transactions that involve at least one account from the current migration batch. Transactions in this pool are temporarily paused until the migration of the current batch is complete.

4.1.3 Operational Workflow. The dual transaction pool mechanism operates as follows:

1. Initial State: Before the migration window opens, all pending transactions reside in the normal transaction pool.

2. Transaction Reclassification: Upon receiving the new allocation scheme V_{new} and the batched migration plan, the transaction shard performs a reclassification. Let $V_{\text{new}} = B_1 \cup B_2 \cup \dots \cup B_N$, where B_i represents the i -th migration batch. When initiating the migration of batch B_i , the shard identifies all transactions in the normal transaction pool that involve any account in B_i and transfers them to the migration transaction pool. All other transactions remain in the normal transaction pool.

3. New Transaction Routing: During the migration window for batch B_i , any newly initiated transaction is immediately checked. If it involves an account in B_i , it is directly added to the migration transaction pool; otherwise, it is added to the normal transaction pool.

4. Parallel Processing: Throughout the migration window, the transaction shard:

- Continues to process transactions from the normal transaction pool, packaging them into regular blocks via the PBFT consensus protocol.
- Suspends processing of transactions in the migration transaction pool to ensure state consistency during the migration of batch B_i . Among them, the transactions in B_i that no longer belong to the current shard will be taken out and transmitted to the migration transaction pool of the new target shard.
- Allows accounts in batches B_j (where $j > i$) to continue normal transaction processing in their current (source) shards, as they are not yet subject to migration.

5. Post-Migration Recovery: After the migration of batch B_i is completed and finalized, the shard retrieves all transactions from the migration transaction pool and re-adds them to the normal transaction pool. These transactions can then be processed in subsequent blocks, as the migrating accounts are now active in their target shards.

4.2 Batch-wise Account Migration Scheme Generation

To maximize the advantages of transaction execution during the migration phase, the core challenge of the BIND protocol lies in generating an optimized batch-wise account migration plan. This section first formalizes the problem mathematically and then introduces an efficient heuristic algorithm designed to solve it.

4.2.1 Problem Formalization and Optimization Objective. We model the accounts designated for migration and their transaction relationships as a weighted undirected graph $G = (V_m, E, W)$, where:

- V_m is the set of accounts to be migrated.
- E is the set of edges representing transaction relationships between accounts.
- $W(u, v)$ is the weight of edge $(u, v) \in E$, quantifying the transaction intensity between accounts u and v .

The objective is to partition the account set V_m into N ordered, mutually disjoint subsets $B_1, B_2, \dots, B_N$, where B_i represents the i -th migration batch. Each batch is subject to a capacity constraint, such that $|B_i| \leq C$ for all $i \in \{1, \dots, N\}$, where C is the maximum number of accounts per batch.

During the migration process, cross-shard communication costs arise from edges that span different batches. The total weight of all edges in the graph, denoted W_{total} , is constant and can be decomposed as:

$$W_{\text{total}} = W_{\text{internal}} + W_{\text{external}}$$

where W_{internal} is the total weight of edges within batches (intra-batch edges), and W_{external} is the total weight of edges across batches (inter-batch edges). The core rationale for minimizing the total migration cost (i.e., maximizing W_{internal}) stems from the phased nature of batch-wise migration. During the migration process, accounts assigned to different batches temporarily reside on different shards. Specifically, if two frequently interacting accounts u and v are placed in different batches B_i and B_j (where $i < j$), their transactions will incur cross-shard communication overhead during the intermediate migration windows (from window i until window $j - 1$). This is because account u will have already migrated to its target shard after window i , while account v remains on its source shard until window j . The weight $W(u, v)$ quantifies this communication cost.

By maximizing W_{internal} , the total weight of edges within batches, we ensure that accounts with strong transactional ties are migrated together in the same batch, thereby avoiding the creation of cross-shard edges during the migration phase. This strategy front-loads the reduction of cross-shard communication costs, minimizing the performance degradation inherent in incremental, batch-wise migration and ensuring that the system reaps the benefits of the new allocation scheme as early as possible.

Minimizing the total migration cost is equivalent to maximizing the total weight of intra-batch edges W_{internal} . Formally, the optimization objective is:

$$\text{maximize } W_{\text{internal}} = \sum_{k=1}^N \left(\sum_{\substack{u, v \in B_k \\ u < v, (u, v) \in E}} W(u, v) \right)$$

subject to:

$$|B_i| \leq C, \quad \forall i \in \{1, \dots, N\}, \quad \text{and} \quad \bigcup_{i=1}^N B_i = V_m, \quad B_i \cap B_j = \emptyset \quad \forall i \neq j$$

This problem is inherently NP-hard, belonging to the class of graph partitioning problems. While it is theoretically possible to obtain the global optimal solution using Integer Linear Programming (ILP) (see Appendix A.1), the computational complexity becomes prohibitive for large-scale instances encountered in real blockchain networks. Therefore, we designed an efficient heuristic algorithm that produces high-quality solutions in polynomial time.

4.2.2 Reverse Greedy Heuristic Algorithm. To obtain high-quality approximate solutions efficiently, we propose a **reverse greedy heuristic algorithm**. The core intuition is to construct the migration batches in reverse order—from batch B_N down to batch B_1 —prioritizing the preservation of tightly connected account communities. By iteratively identifying and removing the *least cohesive* accounts and assigning them to the *later* batches, the algorithm ensures that the most cohesive communities remain intact and are scheduled for *earlier* migration, thereby front-loading the reduction in cross-shard traffic.

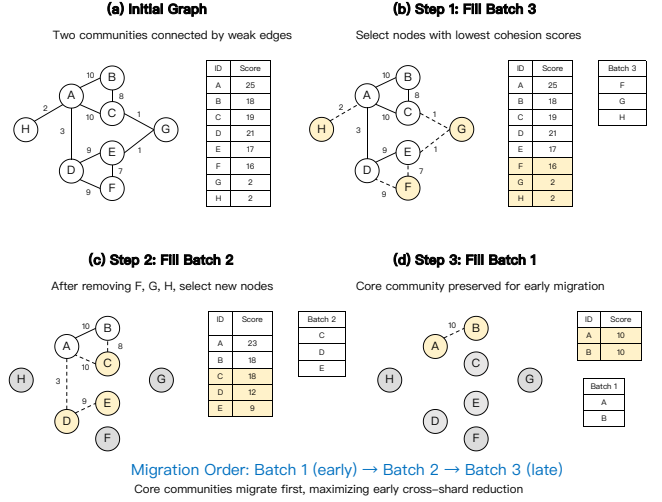


Figure 4: Example of the Reverse Greedy Batching Algorithm.

The algorithm proceeds as follows:

- (1) **Initialization:** Let $U \leftarrow V_m$ be the set of unassigned accounts. Initialize N empty batches $B_1, B_2, \dots, B_N$.
- (2) **Batch Construction (Reverse Order):** For each batch B_k , where k decreases from N to 1:
 - **Calculate Cohesion Scores:** For each account $u \in U$, compute its cohesion score $S(u)$, defined as the sum of edge weights connecting u to all other unassigned accounts:

$$S(u) = \sum_{\substack{v \in U, v \neq u \\ (u, v) \in E}} W(u, v)$$
 A lower cohesion score indicates that an account is less integrated with the remaining community.
 - **Greedy Selection:** Iteratively select the account $u^* \in U$ with the **minimum** cohesion score:

$$u^* = \arg \min_{u \in U} S(u)$$
 - **Batch Assignment:** Add the selected account u^* to the current batch B_k and remove it from the unassigned set U :

$$B_k \leftarrow B_k \cup \{u^*\}, \quad U \leftarrow U \setminus \{u^*\}$$
 - **Capacity Check:** Repeat the selection and assignment until $|B_k| = C$ or $U = \emptyset$.
- (3) **Termination:** The algorithm terminates when all accounts have been assigned, producing the ordered partition $B = (B_1, B_2, \dots, B_N)$.

This reverse construction strategy ensures that accounts with weak connections to the remaining community are "peeled off" first and placed in the later batches. Consequently, the core of the graph—comprising tightly connected communities—is preserved and assigned to the earlier batches. This maximizes the reduction of cross-shard communication costs from the very beginning of the migration process, effectively mitigating the delayed optimization effect inherent in batch-wise migration. Figure 4 provides an example of this reverse greedy batching process, demonstrating how the

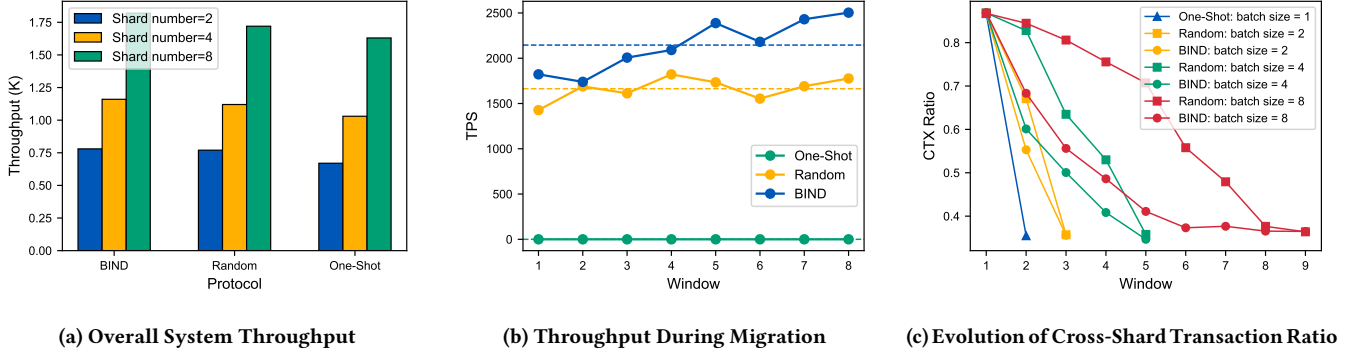


Figure 5: Performance Evaluation: (a) Overall system throughput comparison, (b) Throughput during migration windows, (c) Evolution of cross-shard transaction ratio as migration progresses.

algorithm progressively isolates loosely connected accounts while preserving densely connected cores.

Complexity Analysis: For each batch, the algorithm iterates over at most C accounts. For each account selection, it computes cohesion scores for all remaining accounts in U and selects the minimum, which takes $O(|U| \cdot d)$ time, where d is the average degree of the graph. Given N batches, the overall time complexity is $O(N \cdot C \cdot |V_m| \cdot d)$, which is polynomial and practical for large-scale networks.

5 Implementation and Evaluation

5.1 Implementation

To validate the effectiveness of the BIND protocol, we implemented **BINDChain**, a blockchain sharding system prototype built on BlockEmulator [7]. BINDChain adopts an architecture consisting of one Beacon Shard and multiple transaction shards, integrating all key components of the BIND protocol, including the PBFT consensus algorithm, the dual transaction pool architecture, and the reverse greedy batching algorithm. To coordinate cross-shard transactions, we adopted the relay mechanism from Monoxide [24] and set the block cycle as 5 seconds.

We deployed BINDChain in a distributed environment across three remote servers, each equipped with an Intel(R) Xeon(R) Silver 4314 CPU, 128GB of memory, and configurable network bandwidth up to 20Gbps. Our evaluation used a dataset [30] of 300,000 real historical transactions from Ethereum, representing diverse transaction patterns and account interactions. To simulate the state data of smart contract accounts, the state size of each migrating account was randomly assigned following a logarithmic uniform distribution between 1KB and 10KB, reflecting the heterogeneity of real-world account states.

5.2 Evaluation Setup and Baselines

To comprehensively evaluate the performance of the BIND protocol, we compare it against two baseline mechanisms:

1. One-shot Migration: This represents the traditional approach used in existing systems such as BrokerChain and LB-Chain. All accounts designated for migration are transferred within a single, monolithic migration window. During this period, the entire

transaction pool is locked, and all normal transaction processing is suspended until the migration completes.

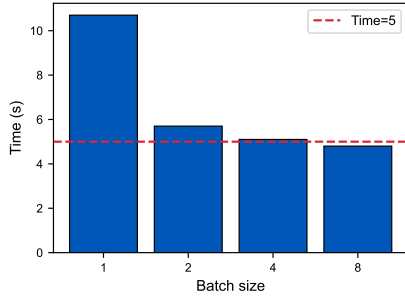
2. Random Batch-wise Migration (Random): This mechanism randomly partitions accounts into multiple batches without considering their interaction patterns. It uses the same dual transaction pool architecture as BIND, allowing it to process transactions for non-migrating accounts during migration windows. However, it lacks an optimized batching strategy, serving as a control to isolate the impact of BIND’s intelligent account selection algorithm.

Our evaluation focuses on three key performance metrics: (1) overall system throughput and throughput during migration windows, (2) the evolution of cross-shard transaction ratio as migration progresses, and (3) the time overhead of each migration window under varying network conditions.

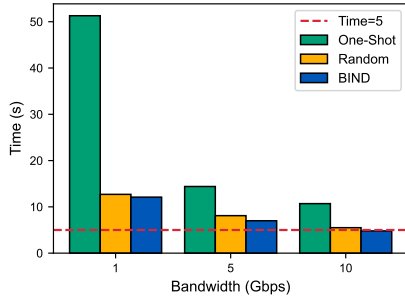
5.2.1 System Throughput. We first evaluate the impact of BIND on system throughput, measuring both overall throughput (across the entire epoch) and throughput specifically during the migration phase.

Overall Throughput: Figure 5a demonstrates that BIND not only prevents service disruption but also significantly improves overall system throughput. In a configuration with 8 shards, BIND achieves 12% higher throughput than the One-shot baseline and 5% higher than the Random baseline. The substantial improvement over One-shot is expected, as BIND processes transactions concurrently with migration, whereas One-shot completely halts all transaction processing. The improvement over Random, despite both mechanisms using the same dual transaction pool architecture, highlights the effectiveness of BIND’s intelligent batching algorithm. By prioritizing the migration of cohesive account communities, BIND more rapidly reduces cross-shard transaction overhead, freeing up system resources and thus boosting overall throughput.

Throughput During Migration: Figure 5b isolates the performance during the migration phase itself. BIND’s throughput during migration windows is substantially higher than both baselines, averaging 29.2% more than Random across 8 migration windows. This is because BIND’s intelligent batching not only allows non-migrating accounts to transact normally but also reduces the cross-shard communication overhead by keeping frequently interacting accounts together. In stark contrast, the One-shot baseline shows zero throughput during its single migration window, as it



(a) Migration Time per Window vs. Number of Batches



(b) Migration Time vs. Network Bandwidth (8 Batches)

Figure 6: Analysis of Migration Window Time Overhead.

completely halts all transaction processing. These results confirm that BIND significantly enhances system availability and performance during the critical migration phase, a period when traditional systems experience severe degradation.

5.2.2 Cross-Shard Transaction Ratio. To evaluate the effectiveness of our reverse greedy batching algorithm, we measured the cross-shard transaction (CTX) ratio as migration progresses through successive windows. The CTX ratio quantifies the proportion of transactions that require inter-shard communication, directly reflecting the optimization effectiveness of the reallocation.

As shown in Figure 5c, while a one-shot migration immediately reduces the CTX ratio to its target of 32% (the ideal state under the new allocation scheme), batch-wise migration introduces a phased reduction. After the first of 8 migration windows, BIND reduces the CTX ratio to 67%, whereas the Random method only achieves 84%. This demonstrates BIND’s ability to *front-load* the benefits of reallocation. By preserving high-cohesion account communities and migrating them early, BIND ensures that the most impactful optimization occurs in the earliest stages, significantly outperforming random selection in every window.

By the midpoint of the migration process (after 4 windows), BIND has already reduced the CTX ratio to 41%, compared to 71% for Random. This sustained advantage minimizes the performance penalty introduced by the batching process itself. Overall, BIND reduces the average cross-shard transaction cost by 24.1% compared to Random. These results validate that BIND’s intelligent batching strategy effectively mitigates the delayed optimization effect

inherent in phased migration, ensuring that the system rapidly approaches the performance benefits of the new allocation scheme.

5.2.3 Migration Window Time Overhead. Finally, we evaluate the time overhead of each migration window, a critical metric for ensuring that the migration process does not disrupt normal block generation cycles. The goal is to keep this overhead comparable to a regular block generation time, thereby achieving a seamless migration experience.

Impact of Batch Count: Figure 6a shows that the time required per migration window decreases as the total number of batches increases. This is because more batches mean fewer accounts (and correspondingly less state data) per batch, which directly reduces the consensus time required for each migration block. With 4 batches, the migration window time is already close to the duration of a regular block generation cycle. With 8 batches, it falls well below this threshold, confirming that BIND can effectively control the duration of each migration step to avoid any noticeable disruption to ongoing transaction processing.

Impact of Network Bandwidth: Figure 6b explores the impact of network bandwidth on migration time, a crucial factor in real-world deployments. BIND consistently and substantially outperforms the One-shot baseline across all bandwidth settings. The advantage is most pronounced in low-bandwidth (1Gbps) environments, where BIND’s migration time is only 23.6% of One-shot’s. This highlights BIND’s high practical value in resource-constrained, real-world networks where bandwidth is a bottleneck. Even in high-bandwidth (10Gbps) scenarios, BIND is still more than twice as fast, requiring only 39.3% of the time needed by One-shot. The performance of BIND and Random are similar in this dimension, as both benefit from the same non-disruptive architecture and small batch sizes. These results prove BIND’s efficiency and adaptability across diverse network conditions, making it a robust solution for production blockchain systems.

6 Conclusion

Account migration is essential for optimizing sharded blockchain performance, yet existing approaches sacrifice system availability during migration. This paper presents BIND, a batch-wise migration protocol that eliminates service disruptions while efficiently reallocating accounts. BIND’s dual transaction pool architecture enables continuous processing for non-migrating accounts, while the reverse greedy batching algorithm front-loads cross-shard communication reduction by prioritizing cohesive account communities. Experiments confirm BIND as a practical solution for large-scale, non-disruptive account migration in production sharded blockchains. Future work will explore adaptive batching for evolving transaction patterns and extensions to heterogeneous sharding architectures.

Acknowledgments

Supported by the Open Fund of National Key Laboratory of Parallel and Distributed Computing (PDL) NO.WDZC20255290109, Shanghai Key Laboratory of Trusted Data Circulation and Governance, and Web3, and the National Natural Science Foundation of China (grant NO. 62372293).

References

- [1] Miguel Castro, Barbara Liskov, et al. 1999. Practical byzantine fault tolerance. In *OsDI*, Vol. 99. 173–186.
- [2] Qinde Chen, Huawei Huang, Zhaokang Yin, Guang Ye, and Qinglin Yang. 2024. Broker2earn: Towards maximizing broker revenue and system liquidity for sharded blockchains. In *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*. IEEE, 251–260.
- [3] Etherscan. 2025. Ethereum Blocks. <https://etherscan.io/blocks> Accessed: 2025-09-06.
- [4] Zicong Hong, Song Guo, Peng Li, and Wuhui Chen. 2021. Pyramid: A layered sharding blockchain system. In *IEEE INFOCOM 2021-IEEE Conference on Computer Communications*. IEEE, 1–10.
- [5] Huawei Huang, Yue Lin, and Zibin Zheng. 2024. Account Migration across Blockchain Shards using Fine-tuned Lock Mechanism. In *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*. IEEE, 271–280.
- [6] Huawei Huang, Xiaowen Peng, Jianzhou Zhan, Shenyang Zhang, Yue Lin, Zibin Zheng, and Song Guo. 2022. Brokerchain: A cross-shard blockchain protocol for account/balance-based state sharding. In *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 1968–1977.
- [7] Huawei Huang, Guang Ye, Qinglin Yang, Qinde Chen, Zhaokang Yin, Xiaofei Luo, Jianru Lin, Jian Zheng, Taotao Li, and Zibin Zheng. 2025. Blockemulator: An emulator enabling to test blockchain sharding protocols. *IEEE Transactions on Services Computing* (2025).
- [8] Junqin Huang, Linghe Kong, Guanjie Cheng, Qiao Xiang, Guihai Chen, Gang Huang, and Xue Liu. 2024. Advancing Web 3.0: Making Smart Contracts Smarter on Blockchain. In *Proceedings of the ACM Web Conference 2024*. 1549–1560.
- [9] Daiki Ito, Yuta Takata, Hiroshi Kumagai, and Masaki Kamizono. 2024. Investigations of top-level domain name collisions in blockchain naming services. In *Proceedings of the ACM Web Conference 2024*. 2926–2935.
- [10] Shan Jiang, Jiannong Cao, Cheung Leong Tung, Yuqin Wang, and Shan Wang. 2024. Sharon: Secure and efficient cross-shard transaction processing via shard rotation. In *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*. IEEE, 2418–2427.
- [11] Eleftherios Kokoris-Kogias, Philipp Jovanovic, Linus Gasser, Nicolas Gailly, Ewa Syta, and Bryan Ford. 2018. Omniledger: A secure, scale-out, decentralized ledger via sharding. In *2018 IEEE symposium on security and privacy (SP)*. IEEE, 583–598.
- [12] Michał Król, Onur Ascigil, Sergi Rene, Alberto Sonnino, Mustafa Al-Bassam, and Etienne Rivière. 2021. Shard scheduler: Object placement and migration in sharded account-based blockchains. In *Proceedings of the 3rd ACM Conference on Advances in Financial Technologies*. 43–56.
- [13] Canlin Li, Huawei Huang, Yetong Zhao, Xiaowen Peng, Ruijie Yang, Zibin Zheng, and Song Guo. 2022. Achieving scalability and load balance across blockchain shards for state sharding. In *2022 41st International Symposium on Reliable Distributed Systems (SRDS)*. IEEE, 284–294.
- [14] Mingzhe Li, You Lin, Jin Zhang, and Wei Wang. 2023. CoChain: High concurrency blockchain sharding via consensus on consensus. In *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*. IEEE, 1–10.
- [15] Mingzhe Li, Wei Wang, and Jin Zhang. 2023. LB-Chain: Load-balanced and low-latency blockchain sharding via account migration. *IEEE Transactions on Parallel and Distributed Systems* 34, 10 (2023), 2797–2810.
- [16] Pengze Li, Mingxuan Song, Mingzhe Xing, Zhen Xiao, Qiuyu Ding, Shengjie Guan, and Jieyi Long. 2024. SPRING: Improving the Throughput of Sharding Blockchain via Deep Reinforcement Learning Based State Placement. In *Proceedings of the ACM on Web Conference 2024*. 2836–2846.
- [17] Xintong Ling, Rui Jiang, Weihang Cao, Mingkai Chen, Jiaheng Wang, Zhi Ding, and Xiqi Gao. 2025. Exploring MEC Server Strategy in Blockchain Networks: Mining For Mobile Users or For Self. *IEEE Transactions on Mobile Computing* (2025).
- [18] Loi Luu, Viswesh Narayanan, Chaodong Zheng, Kunal Baweja, Seth Gilbert, and Prateek Saxena. 2016. A secure sharding protocol for open blockchains. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security*. 17–30.
- [19] Liyuan Ma, Dengcheng Hu, Xiulong Liu, Hao Xu, Jianrong Wang, and Keqiu Li. 2025. AIGC-CM: An Efficient and Scalable Blockchain Solution for AIGC Copyright Management. In *IEEE INFOCOM 2025-IEEE Conference on Computer Communications*. IEEE, 1–10.
- [20] Jiahao Qi, Dian Ding, Yue Pan, Han Zhang, Lei Tian, Yi-Chao Chen, Jiong Lou, Jiadi Yu, Qiaoling Xiao, Jie Li, et al. 2025. Breaking the Mainchain Barrier of Blockchain Sharding Architecture for Federated Learning. In *2025 IEEE/ACM 33rd International Symposium on Quality of Service (IWQoS)*. IEEE, 1–6.
- [21] Jiahao Qi, Feilong Lin, Zhongyu Chen, Changbing Tang, Riheng Jia, and Minglu Li. 2022. High-quality model aggregation for blockchain-based federated learning via reputation-motivated task participation. *IEEE Internet of Things Journal* 9, 19 (2022), 18378–18391.
- [22] Yixiao Teng, Jiamei Lv, Ziping Wang, Yi Gao, and Wei Dong. 2025. TimeChain: A Secure and Decentralized Off-chain Storage System for IoT Time Series Data. In *Proceedings of the ACM on Web Conference 2025*. 3651–3659.
- [23] Lei Tian, Feilong Lin, Jiahao Gan, Jiahao Qi, Riheng Jia, Zhonglong Zheng, Minglu Li, and Xuemin Shen. 2025. Enhancing Security in Parallel Federated Learning with Sharded Blockchain for Internet of Vehicles. *IEEE Transactions on Vehicular Technology* (2025), 1–16. doi:10.1109/TVT.2025.3593493
- [24] Jiaping Wang and Hao Wang. 2019. Monoxide: Scale out blockchains with asynchronous consensus zones. In *16th USENIX symposium on networked systems design and implementation (NSDI 19)*. 95–112.
- [25] Jiajing Wu, Kaixin Lin, Dan Lin, Bozhao Zhang, Zhiying Wu, and Jianzhong Su. 2025. Safeguarding blockchain ecosystem: Understanding and detecting attack transactions on cross-chain bridges. In *Proceedings of the ACM on Web Conference 2025*. 4902–4912.
- [26] Jie Xu, Yulong Ming, Zihan Wu, Cong Wang, and Xiaohua Jia. 2024. X-Shard: Optimistic Cross-Shard Transaction Processing for Sharding-Based Blockchains. *IEEE Transactions on Parallel and Distributed Systems* (2024).
- [27] Minghui Xu, Hechuan Guo, Ye Cheng, Chunchi Liu, Dongxiao Yu, and Xiuzhen Cheng. 2025. EC-Chain: Cost-Effective Storage Solution for Permissionless Blockchains. In *IEEE INFOCOM 2025-IEEE Conference on Computer Communications*. IEEE, 1–10.
- [28] Mahdi Zamani, Mahnush Movahedi, and Mariana Raykova. 2018. Rapidchain: Scaling blockchain via full sharding. In *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*. 931–948.
- [29] Peilin Zheng, Quanqing Xu, Zibin Zheng, Zhiyuan Zhou, Ying Yan, and Hui Zhang. 2021. Meepo: Sharded consortium blockchain. In *2021 IEEE 37th International Conference on Data Engineering (ICDE)*. IEEE, 1847–1852.
- [30] Peilin Zheng, Zibin Zheng, Jiajing Wu, and Hong-Ning Dai. 2020. Xblock-eth: Extracting and exploring blockchain data from ethereum. *IEEE Open Journal of the Computer Society* 1 (2020), 95–106.

A Appendix

A.1 Integer Linear Programming Formulation for Optimal Batching

This appendix provides the formal Integer Linear Programming (ILP) model for the batch-wise account migration problem described in Section 4.2.1. The objective is to partition the set of migrating accounts into a predefined number of batches, maximizing the total weight of intra-batch connections. This is equivalent to minimizing the communication cost incurred by splitting highly connected accounts across different batches.

A.1.1 Model Parameters.

- V_m : The set of accounts to be migrated.
- W_{uv} : The weight of the edge between account u and account v ($u, v \in V_m$), representing their interaction intensity.
- N : The total number of migration batches.
- C : The maximum capacity (number of accounts) for each batch.

A.1.2 Decision Variables.

- We define two sets of binary variables:
- x_{uk} : A binary variable that equals 1 if account $u \in V_m$ is assigned to batch $k \in \{1, \dots, N\}$, and 0 otherwise.
 - y_{uvk} : A binary variable that equals 1 if both accounts u and v ($u, v \in V_m, u < v$) are assigned to the same batch k , and 0 otherwise. This variable is used to linearize the quadratic objective function.

A.1.3 Objective Function. The goal is to maximize the sum of weights of all edges whose endpoints are within the same batch. This is formulated as:

$$\text{maximize} \quad \sum_{k=1}^N \sum_{u,v \in V_m, u < v} W_{uv} \cdot y_{uvk}$$

A.1.4 Constraints. The optimization is subject to the following constraints:

- (1) **Assignment Constraint:** Each account must be assigned to exactly one batch.

$$\sum_{k=1}^N x_{uk} = 1, \quad \forall u \in V_m$$

- (2) **Capacity Constraint:** The number of accounts in each batch cannot exceed the maximum capacity C .

$$\sum_{u \in V_m} x_{uk} \leq C, \quad \forall k \in \{1, \dots, N\}$$

- (3) **Linearization Constraints:** These constraints link the y_{uvk} variables to the x_{uk} variables, ensuring that $y_{uvk} = 1$ if and only if both $x_{uk} = 1$ and $x_{vk} = 1$.

$$y_{uvk} \leq x_{uk}, \quad \forall u, v \in V_m, u < v, \forall k \in \{1, \dots, N\} \quad (1)$$

$$y_{uvk} \leq x_{vk}, \quad \forall u, v \in V_m, u < v, \forall k \in \{1, \dots, N\} \quad (2)$$

$$y_{uvk} \geq x_{uk} + x_{vk} - 1, \quad \forall u, v \in V_m, u < v, \forall k \in \{1, \dots, N\} \quad (3)$$

- (4) **Variable Type Constraints:** All decision variables are binary.

$$x_{uk} \in \{0, 1\}, \quad \forall u \in V_m, \forall k \in \{1, \dots, N\} \quad (4)$$

$$y_{uvk} \in \{0, 1\}, \quad \forall u, v \in V_m, u < v, \forall k \in \{1, \dots, N\} \quad (5)$$

While this ILP formulation provides a precise model for finding the global optimum, the number of variables and constraints grows polynomially with the number of accounts and batches, making it computationally intractable for large-scale instances. This justifies the use of the efficient heuristic algorithm presented in the main body of the paper.

A.2 Security Analysis

The security of an account migration protocol is paramount to maintaining the integrity and reliability of a sharded blockchain. In this section, we analyze the robustness of the BIND protocol against several potential security threats, including double-spending attacks and state inconsistency.

A.2.1 Prevention of Double-Spending and State Inconsistency. A primary security challenge during account migration is the risk of double-spending, where an attacker attempts to execute conflicting transactions from the same account on both the source and target shards simultaneously. BIND effectively mitigates this threat through its dual transaction pool architecture and strict state transition mechanism.

- **Transaction Isolation:** When an account is designated for migration in batch B_i , all existing and new transactions involving that account are immediately moved to the *migration transaction pool*. This pool is locked and its transactions are paused until the migration of batch B_i is complete. Concurrently, the *normal transaction pool* only processes transactions for non-migrating accounts. This strict isolation ensures that no transactions for a migrating account can be processed on the source shard during its migration window, thus preventing any state changes that could lead to double-spending.

- **Atomic State Transition:** The migration of an account's state is recorded in a special *migration block*. This block is subject to the same PBFT consensus as a regular block. The account's state is only considered officially moved to the target shard after the migration block achieves consensus and is finalized on the source shard's ledger. Before this finalization, the account remains inactive on the target shard. After finalization, the account is deactivated on the source shard. This atomic, consensus-driven state transition guarantees that an account exists in an active, transactable state on only one shard at any given time, thereby preventing state inconsistencies.

A.2.2 Integrity of the Migration Plan. The integrity of the batched migration plan is critical, as a malicious plan could degrade network performance or target specific accounts. BIND ensures this integrity through the security of the Beacon Shard.

- **Consensus on Migration Plan:** The generation of the new allocation scheme and the subsequent batched migration plan occurs within the Beacon Shard. This plan is not dictated by a single entity but is agreed upon by the consensus nodes of the Beacon Shard using the PBFT protocol. To manipulate the migration plan, an attacker would need to compromise more than one-third of the nodes in the Beacon Shard, which is the standard security assumption for Byzantine fault-tolerant systems. This ensures that any generated and distributed migration plan is valid and has been agreed upon by a supermajority of honest nodes.

A.3 Discussion

The experimental results demonstrate that BIND successfully addresses the fundamental trade-off between migration efficiency and system availability in sharded blockchains. By enabling continuous transaction processing and intelligently optimizing the batching strategy, BIND achieves:

- **High Availability:** BIND maintains 100% transaction processing capability for non-migrating accounts throughout the migration process, eliminating the service disruptions inherent in traditional one-shot approaches.
- **Efficient Optimization:** BIND's reverse greedy algorithm front-loads the reduction of cross-shard communication costs, ensuring that the system rapidly approaches the performance benefits of the new allocation scheme. By isolating only the transactions directly affected by the current migration batch, BIND minimizes service disruption and maintains high throughput, enabling the sharded system to operate seamlessly during the entire migration process.
- **Scalability:** BIND's migration window time remains comparable to regular block generation cycles, and it scales effectively across diverse network bandwidths, making it suitable for real-world deployment. The transaction pool switching operations are highly efficient because transaction pools are maintained entirely in memory. The overhead of these operations is negligible and does not significantly impact the transaction processing capacity of the shard.